

Android Data Storage

SQLite Database

- Sqlite is a very lightweight database which comes with Android OS.
- Once a database is created successfully its located in **data/data//databases/** accessible from Android Device Monitor.
- SQLite is a typical **relational database**, containing tables (which consists of rows and columns), indexes etc.

SQLiteOpenHelper

- Android has features available to handle changing database schemas, which mostly depend on using the SQLiteOpenHelper class. **SQLiteOpenHelper** is designed to get rid of two very common problems.
 - When the application runs the first time - At this point, we do not yet have a database. So we will have to create the tables, indexes, starter data, and so on.
 - When the application is upgraded to a newer schema - Our database will still be on the old schema from the older edition of the app. We will have option to alter the database schema to match the needs of the rest of the app.

SQLiteOpenHelper

SQLiteOpenHelper subclass that manages database creation and version management

```
class MyDatabaseHelper(context: Context) : SQLiteOpenHelper(context,
    DATABASE_NAME, null, DATABASE_VERSION) {

    override fun onCreate(db: SQLiteDatabase) {
        // Create tables
    }

    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int,
        newVersion: Int) {
        // Upgrade database if needed
    }
}
```

Opening and Closing Android SQLite Database Connection

Before performing any database operations like insert, update, delete records in a table, first open the database connection by calling

getWritableDatabase() method as shown below:

```
val dbHelper = MyDatabaseHelper(context)
val db = dbHelper.getWritableDatabase
```

```
// Perform database operations...
```

```
db.close()
```

It's important to close the database after you're done with it to release resources and avoid memory leaks.

Inserting new Record into Android SQLite database table

```
val values = ContentValues().apply {  
    put(COLUMN_NAME, "Sok Chan")  
    put(COLUMN_AGE, 35)  
}
```

```
val status = db.insert(TABLE_NAME, null, values)
```

The insert() method returns the row ID of the newly inserted row, or -1 if an error occurred.

Query Data

```
val projection = arrayOf(COLUMN_ID, COLUMN_NAME, COLUMN_AGE)
```

```
val selection = "$COLUMN_AGE > ?"
```

```
val selectionArgs = arrayOf("25")
```

```
val cursor = db.query(
```

```
    TABLE_NAME,
```

```
    projection,
```

```
    selection,
```

```
    selectionArgs,
```

```
    null,
```

```
    null,
```

```
    null
```

```
)
```

Query Data

```
with(cursor) {  
    while (moveToNext()) {  
        val itemId = getLong(getColumnIndexOrThrow(COLUMN_ID))  
        val itemName = getString(getColumnIndexOrThrow(COLUMN_NAME))  
        val itemAge = getInt(getColumnIndexOrThrow(COLUMN_AGE))  
        // Use the data...  
    }  
}  
cursor.close()
```


Updating Record in Android SQLite database table

```
val values = ContentValues().apply {  
    put(COLUMN_NAME, "Sok Chan")  
}
```

```
val selection = "$COLUMN_NAME LIKE ?"  
val selectionArgs = arrayOf("Sok Chan")
```

```
val count = db.update(  
    TABLE_NAME,  
    values,  
    selection,  
    selectionArgs  
)
```

Android SQLite – Deleting a Record

```
val selection = "$COLUMN_AGE < ?"
```

```
val selectionArgs = arrayOf("25")
```

```
val deletedRows = db.delete(TABLE_NAME, selection,  
selectionArgs)
```

Share Preference

- **SharedPreferences** allow you to save and retrieve data in the form of key, value pair.
- Android stores Shared Preferences settings as XML file in **shared_prefs** folder under DATA/data/{application package} directory.
- **SharedPreferences** is application specific, i.e. the data is lost on performing one of the following options:
 - on uninstalling the application
 - on clearing the application data (through Settings)

Accessing Shared Preferences

- **getPreferences()** : used from within your Activity, to access activity-specific preferences
- **getSharedPreferences()** : used from within your Activity (or other application Context), to access application-level preferences
- **getDefaultSharedPreferences()** : used on the PreferenceManager, to get the shared preferences that work in concert with Android's overall preference framework

Initialization Sharepreference

We need an editor to edit and save the changes in shared preferences.

The following code can be used to get the shared preferences.

```
val sharedPreferences = getSharedPreferences("app_preferences",  
Context.MODE_PRIVATE)
```

Storing Data, Sharepreference

```
val sharedPreferences =  
    getSharedPreferences("PREFERENCE_NAME",Context.MODE_PRIVATE)  
  
var editor = sharedPreferences.edit()  
  
editor.putString("USERNAME","Dara Heng")  
  
editor.putString("EMAIL","daraheng@gmail.com")  
  
editor.apply()
```


Retrieving Data, Sharepreference

```
sharedPreference.getString("USERNAME","")
```

```
sharedPreference. getString("EMAIL","")
```

Clearing or Deleting Data , Sharepreference

```
val editor = sharedPreferences.edit()
```

```
editor.remove("USERNAME")
```

```
Editor.remove("EMAIL")
```

```
editor.apply()
```

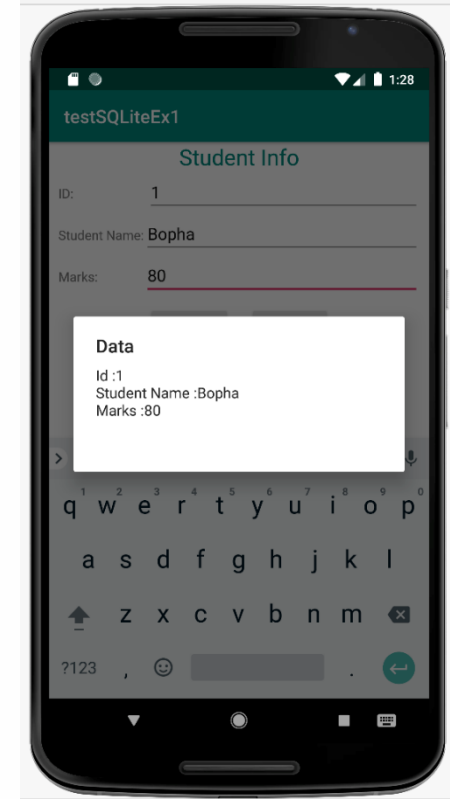
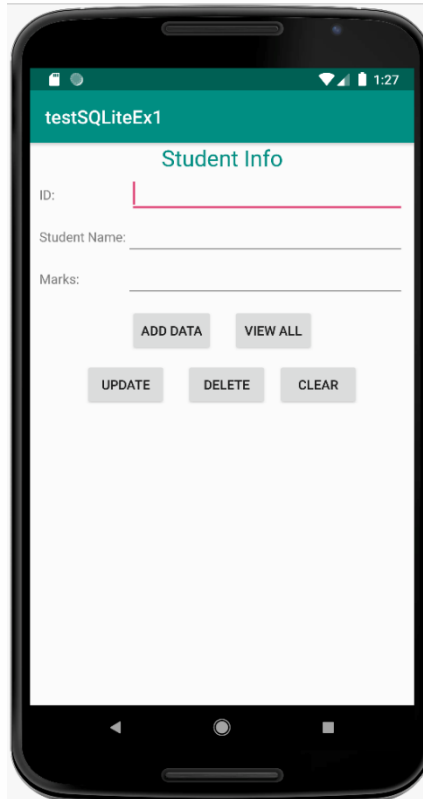
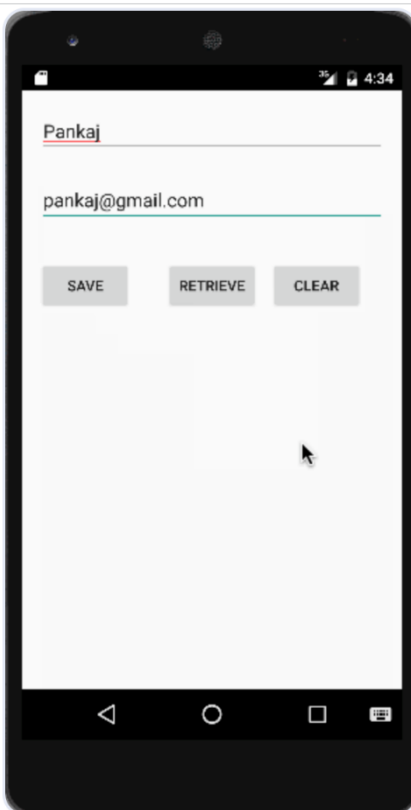
```
// below line will clear the data in shared prefs.
```

```
editor.clear()
```

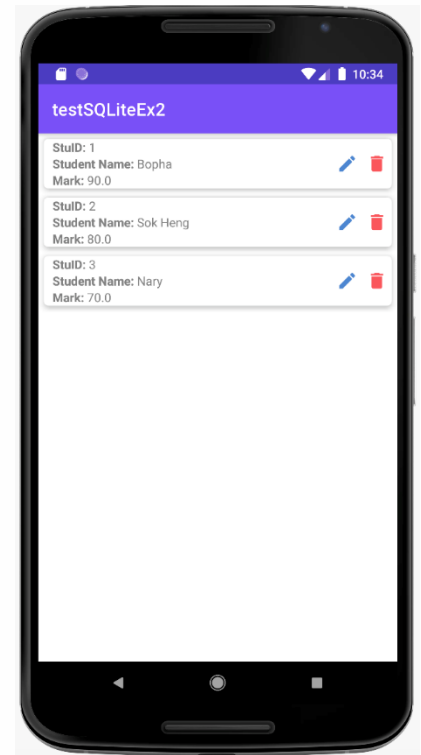
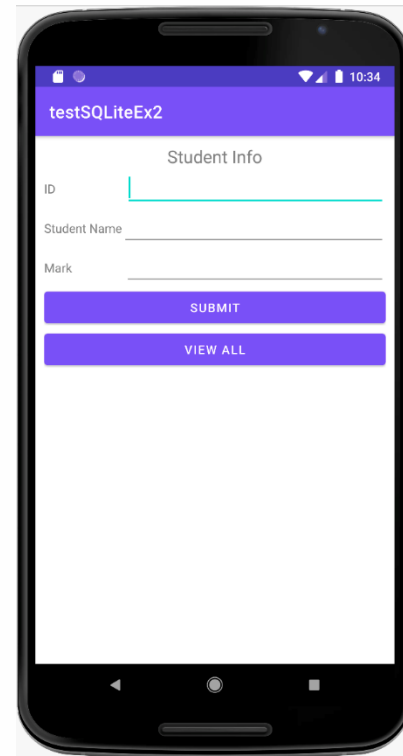
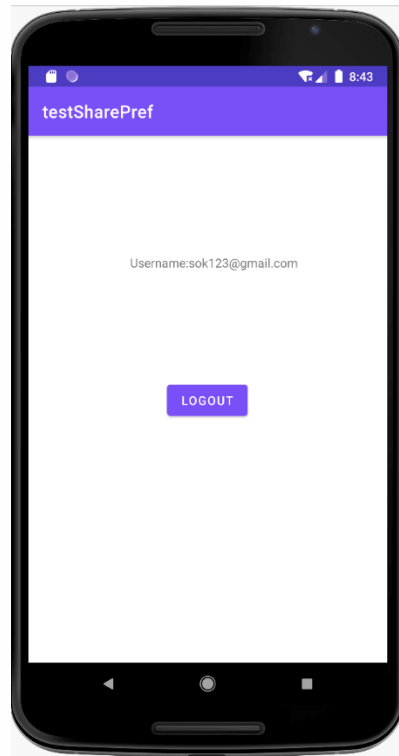
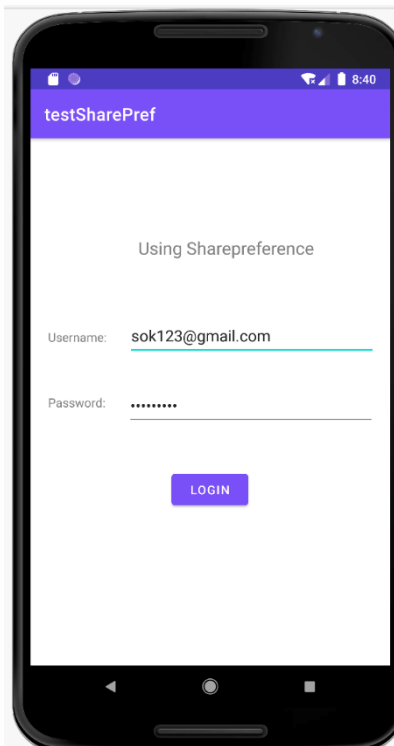
```
// below line will apply empty data to shared prefs.
```

```
editor.apply()
```

Exercises



Exercises



Thank you